4-bit Unsigned Array Multiplier: RTL to GDSII on SkyWater SKY130

Bhargav (2023112014) • Nitin (2023112026)

Digital VLSI Design, Assignment 1 • October 2026

Repository: `https://github.com/ntini773/DVD_A1`

1 Introduction

This report follows a 4-bit × 4-bit unsigned array multiplier (8-bit product) through the complete open-source chip design flow on the SkyWater SKY130 130 nm process with the `sky130_fd_sc_hd` standard-cell library (*foundry, digital, standard cell, high density*). Each part of the flow answers one question:

- **Part A (RTL design and verification):** is the logic correct? The design is written in Verilog and simulated against every possible input.
- **Part B (synthesis):** what real gates does it use? Yosys converts the Verilog into a netlist of SKY130 library cells.
- **Part C (static timing analysis):** how fast can it run? OpenSTA adds up the delays of the gates on every path.
- **Part D (physical design):** does it still work with real wires, and can it be manufactured? LibreLane places and routes the cells into a layout (GDSII), checks it, and re-measures timing with real wires.

The parts are linked:

- Part B re-uses the Part A testbench to check its netlist.
- Part C analyses the Part B netlist.
- Part C's minimum clock period sets the first clock period tried in Part D.
- Part D's cell count and speed are compared with Parts B and C.

Table 1 lists the tools used. All source files, scripts, generated reports, the final layout (GDSII) and the exact command for every step (`README.md`) are in our repository: `https://github.com/ntini773/DVD_A1`.

Stage	Tool and version
Simulation, waveforms	Icarus Verilog (`-g2012`), GTKWave
Logic synthesis	Yosys 0.69+ with ABC
Static timing analysis	OpenSTA 3.1.0
RTL-to-GDSII flow	LibreLane v3.1.0.dev3, Classic flow (OpenROAD, Magic, KLayout, Netgen)
PDK and library	sky130A, `sky130_fd_sc_hd`; typical corner `tt_025C_1v80` (typical transistors, 25 °C, 1.80 V) for Parts B and C
Environment	Ubuntu 22.04 host (Parts A and B); IIC-OSIC-TOOLS Docker container (Parts C and D)

Table 1: Tools used in each stage of the flow.

2 Design and Verification (Part A)

2.1 How the array multiplier works

Binary multiplication is shift-and-add. For each bit b_i of B there is one *partial product* $pp_i = A \wedge b_i$: a copy of A if $b_i = 1$, or zero if $b_i = 0$, shifted left by i places. Each bit of a partial product is one AND gate, $pp_i[j] = a[j] \cdot b[i]$, so the four partial products need $4 \times 4 = 16$ AND gates. Adding the four partial products gives the 8-bit product.

The hardware adds them one row at a time (Table 2):

- Row 0 is AND gates only. Its lowest bit is already the final bit $p[0]$, and its other three bits pass down.
- Rows 1 to 3 are each a 4-bit *ripple-carry adder*: 4 full adders in a chain, each passing its carry to the next. Row i adds pp_i to the 4 bits passed down from the row above, giving $3 \times 4 = 12$ full adders in total.
- The lowest sum bit of each row is a finished product bit ($p[1]$, $p[2]$, $p[3]$). Row 3 also produces the top four bits $p[7{:}4]$.

Row	Inputs to adder j	Outputs
0	AND gates only	$p[0] = pp_0[0]$; $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
1	$pp_1[j]$, $r_0[j]$, $c_1[j]$	$s_1[j]$, $c_1[j{+}1]$; $p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3{:}1]\}$
2	$pp_2[j]$, $r_1[j]$, $c_2[j]$	$s_2[j]$, $c_2[j{+}1]$; $p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3{:}1]\}$
3	$pp_3[j]$, $r_2[j]$, $c_3[j]$	$s_3[j]$, $c_3[j{+}1]$; $p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4], s_3[3{:}1]\}$

Table 2: Connections of the array ($j = 0 \ldots 3$; the carry into each row is $c_i[0] = 0$).

2.2 RTL implementation

The full adder (`rtl/full_adder.v`) is written with two `assign` statements:

$$s = a \oplus b \oplus c_{in}, \qquad c_{out} = ab + a\,c_{in} + b\,c_{in}.$$

The sum is 1 when an odd number of inputs are 1 (XOR). The carry is the *majority* function: it is 1 when two or more inputs are 1.

`rtl/mult_array.v` has the required ports:

- `clk`: rising-edge clock.
- `rst_n`: asynchronous, active-low reset. When it is 0, all registers clear to 0 at once, without waiting for the clock.
- `a[3:0]`, `b[3:0]`: the inputs.
- `p[7:0]`: the registered product.

Inside the module:

1. On every rising edge, the inputs are captured in 4-bit registers `a_r` and `b_r` (8 flip-flops).
2. The partial products are `pp`i `= a_r & {4{b_r[`i`]}}` (16 AND gates).
3. Three `generate` loops each place 4 `full_adder` instances (`row1.fa`, `row2.fa`, `row3.fa`), which gives 12 instances.
4. The result `{c3[4], s3[3:0], s2[0], s1[0], pp0[0]}` is registered into `p` (8 flip-flops).

That is 16 flip-flops in total, and the `*` operator is not used. Because both the inputs and the output are registered, a value applied before clock edge 1 is captured at edge 1, and its product is captured at edge 2. The *latency* is therefore 2 clock cycles. The *throughput* is one new product per cycle, because a new input pair can enter at every edge while the previous one is still being computed. Fig. 1 shows the structure.

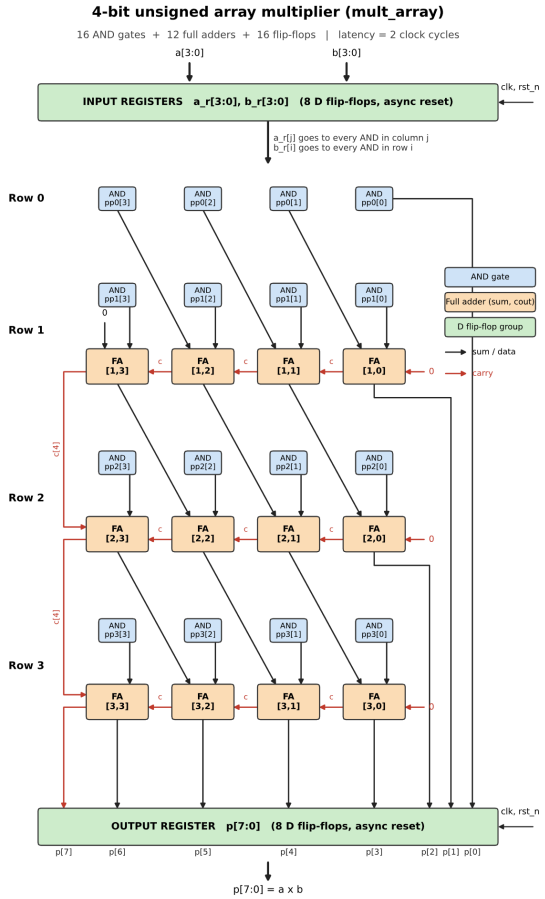


Figure 1: Block diagram: 8 input flip-flops, 16 AND gates, 12 full adders in three ripple-carry rows (red arrows are carries), and 8 output flip-flops.

2.3 Testbench

`tb/tb_mult.v` is *exhaustive* (it tries every input) and *self-checking* (it computes the expected answer and compares automatically). Table 3 lists each requirement and how the testbench meets it.

Each vector takes 3 clock cycles (30 ns), so the run ends at $20\text{ ns} + 256 \times 30\text{ ns} = 7700\text{ ns}$. This matches the simulator's `$finish` time of 7 700 000 ps.

Requirement	Implementation
10 ns clock; reset for 2 cycles	`always #5 clk = ~clk`; `rst_n` = 0 until the falling edge after the second rising edge (20 ns).
All 256 input pairs	Nested loops `i, j` = 0…15.
Respect the 2-cycle latency	Inputs change on a falling edge; the testbench waits two rising edges and checks `p` on the next falling edge, away from any clock edge.
Compare with `a * b`	`expected = i * j`; the `*` operator is used only here.
Report	One `MISMATCH` line per error, then exactly one `PASS:/FAIL:` summary line.
Waveform	`$dumpfile("mult.vcd")`.

Table 3: Testbench requirements and how `tb/tb_mult.v` meets them.

2.4 Simulation results and error test

The RTL simulation prints `PASS: 256 vectors checked, 0 errors`, so the design is correct for every input.

A testbench is only useful if it can fail. To check this, `sum` in `full_adder.v` was changed to `a ^ b`, which ignores the carry-in. The testbench then printed `FAIL: 256 vectors checked, 95 errors`; Listing 1 in the Appendix shows the first mismatches. A vector can only fail when some full adder actually receives a carry-in of 1. Many vectors never create an internal carry (for example $a = 0$, $b = 0$, or an operand that is a power of two), which is why the other 161 vectors still pass. The change was then undone and the testbench prints PASS again.

2.5 Hand-worked example

A is the last digit of a roll number plus 5. Roll number 2023112014 ends in 4, so A = 9 = 1001_2, and B = 7 = 0111_2:

```
          1 0 0 1     A = 9
        x 0 1 1 1     B = 7
---------------
          1 0 0 1     pp0 = A AND b0 (b0 = 1)
        1 0 0 1       pp1 = A AND b1 (b1 = 1), shifted 1
      1 0 0 1         pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 0 1 1 1 1 1 1       P = 63
```

Table 4 computes the same product the way the hardware does, row by row, using the connections of Table 2. Both methods give $0011\ 1111_2 = 63 = 0\text{x}3\text{F}$.

Row	pp_i	$+ r_{i-1}$	$= \{c, s\}$	Outputs
0	1001	–	–	$p[0] = 1$, $r_0 = 0100$
1	1001	0100	0 1101	$p[1] = 1$, $r_1 = 0110$
2	1001	0110	0 1111	$p[2] = 1$, $r_2 = 0111$
3	0000	0111	0 0111	$p[3] = 1$, $p[7{:}4] = 0011$

Table 4: Row-by-row evaluation of 9×7 in the array (all values in binary). No full adder produces a carry for this input pair.

2.6 Waveform

Fig. 2 shows this example in simulation:

1. The inputs change to $a = 9$, $b = 7$ at 30 ns.
2. At clock edge 1 (35 ns), the input registers capture them.
3. During the next cycle, the array computes the product.

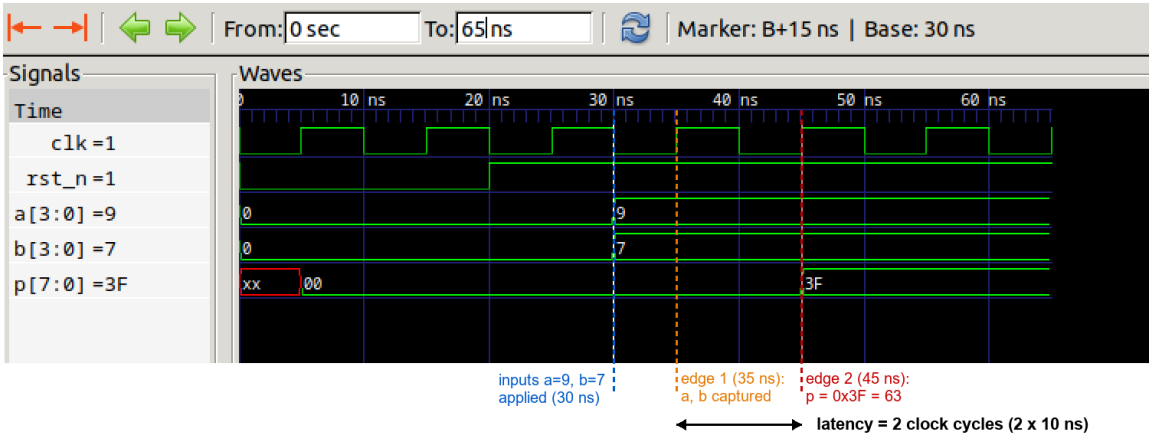


Figure 2: GTKWave view of `clk`, `rst_n`, `a`, `b` and `p` for A = 9, B = 7 (10 ns clock period). The inputs change at 30 ns and are captured at clock edge 1 (35 ns). The product 0x3F = 63 appears on `p` at edge 2 (45 ns), so the latency is 2 clock cycles (20 ns).

4. At edge 2 (45 ns), the output register captures 0x3F = 63.

This shows the 2-cycle latency directly. Before 5 ns, `p` shows `xx` (unknown). `rst_n` starts at 0 without a falling edge, so the asynchronous-reset block first runs at the first clock edge, which then clears `p` to 00. This is a simulation start-up effect only.

3 Synthesis (Part B)

3.1 What synthesis does

Logic synthesis turns the behavioural Verilog into a *netlist*: a list of real cells from the SKY130 library and the wires between them. Each cell is a pre-designed gate whose area and delay are given in the *Liberty* file (`.lib`). Table 5 lists the steps of `synth/synth.tcl`. The key step is `abc`: it does *technology mapping*, choosing the combination of library gates that implements the logic with good area and delay. Because of `-flatten`, ABC sees the whole multiplier as one block of logic rather than 12 separate full adders, so it can optimise across adder boundaries.

Step	Yosys command	Purpose
1	`read_verilog`	Read the two RTL files
2	`synth -top mult_array -flatten`	Generic synthesis; merge the full adders into one module
3	`dfflibmap -liberty $lib`	Map the 16 flip-flops to SKY130 cells
4	`abc -liberty $lib`	Map the logic to SKY130 gates
5	`opt_clean -purge`	Remove unused wires
6	`tee -o ... stat -liberty $lib`	Write the cell and area report
7	`write_verilog -noattr`	Write the gate-level netlist

Table 5: Steps of `synth/synth.tcl`, run with `yosys -c synth/synth.tcl`.

3.2 Results

Table 6 gives the required results. The flip-flops are `dfrtp_1` cells: D flip-flops with an active-low asynchronous reset (the `_1` suffix is the drive strength). The 47 logic cells are:

- 10 XOR/XNOR cells and 3 `maj3` cells. These are the full adders' sum and carry functions.
- 6 AND cells.
- 16 NAND/NOR cells.
- 10 complex gates, such as `a21oi` $= \overline{A_1A_2 + B_1}$ and `o311ai` $= \overline{(A_1 + A_2 + A_3)B_1C_1}$. In these names, "a" is AND, "o" is OR and "i" means an inverted output.
- 2 inverters.

Table 15 in the Appendix lists every cell with its area.

Metric	Value
Total number of cells	63 cells
Number of flip-flops	16 cells (`dfrtp_1`)
Total cell area	740.71 μm^2
Flip-flop area	400.38 μm^2 (16 $\times$ 25.024 μm^2)
Flip-flop area (% of total)	54.05 %
3 most-used cell types	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 8, `a21oi_1` $\times$ 6
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 6: Part B results (from `synth/mult_array_stat.rpt`).

3.3 Gate-level simulation

Synthesis could in principle change the behaviour, for example if a register were mis-mapped. To rule this out, the synthesised netlist was simulated with the *same* testbench, using the SKY130 Verilog cell models (`primitives.v`, `sky130_fd_sc_hd.v`). `-DFUNCTIONAL` selects the logic-only cell models, and `-DUNIT_DELAY=#1` gives every cell a small unit delay (`#1`) so that signals change in a clean order. The result is again `PASS: 256 vectors checked, 0 errors` (Listing 2), so the netlist behaves exactly like the RTL.

4 Timing (Part C)

4.1 Static timing analysis

Static timing analysis (STA) checks speed without simulation. It looks at every path from one flip-flop to the next and adds up the cell delays from the Liberty file.

A path launched at one clock edge must reach the next flip-flop before the following edge, so the *setup* condition is

$$t_{clk\to Q} + t_{logic} + t_{setup} \le T.$$

Here $t_{clk\to Q}$ is the time the launching flip-flop takes to update its output, t_{logic} is the delay through the gates, and t_{setup} is how long the data must be stable before the capturing edge.

The *slack* is the required time ($T - t_{setup}$) minus the arrival time. Positive slack means the path meets timing; negative slack means it fails. The *worst negative slack* (WNS) is the smallest slack in the design, and the path that has it is the *critical path*.

`sta/sta.tcl` performs these steps:

1. Read the typical-corner library and the Part B netlist.
2. Create a clock of period T on `clk`.
3. Set an input delay and an output delay of 1.0 ns. These model logic outside the chip, so the I/O paths are also checked.
4. Report the worst slack and the critical path.

The clock is *ideal*: it arrives at every flip-flop at the same time. There are no wire delays, because the design has not been placed yet.

4.2 Maximum frequency

At $T = 10$ ns the worst slack is +7.396 ns, so the circuit could run 7.396 ns faster:

$$T_{\min} = 10\text{ ns} - 7.396\text{ ns} = 2.604\text{ ns},$$
$$F_{\max} = 1000/2.604 = 384.02\text{ MHz}.$$

Re-running at $T = 2.60$ ns ($T_{\min}$ rounded to 0.01 ns) gives a slack of -0.004 ns. That is effectively zero, which confirms $T_{\min}$; the 0.004 ns comes only from rounding 2.604 ns down to 2.60 ns. Table 7 summarises the results.

Metric	Value
Worst slack at $T = 10$ ns	+7.396 ns
$T_{\min}$	2.604 ns
$F_{\max}$	384.02 MHz
Slack at $T = 2.60$ ns	-0.004 ns (≈ 0 ns)
Critical path start point	`_095_` = input register `a_r[1]`
Critical path end point	`_092_` = output register `p[6]`
Cells on the critical path	10 cells (2 flip-flops + 8 gates)
Latency $= 2 \times T_{\min}$	5.208 ns
Throughput	384.02 million results per second

Table 7: Part C results (OpenSTA, typical corner, pre-layout).

4.3 Critical path

Table 8 lists the critical path cell by cell. The instance names (`_095_`, `_092_`) are generated by Yosys. Tracing them in the netlist shows that they are the flip-flops for `a_r[1]` and `p[6]`.

Identifying each gate. Because Yosys flattened the full adders, the netlist gates carry generated names, not adder names. To find what each gate computes, the netlist was evaluated for all 256 values of (`a_r`, `b_r`). The truth table of every net on the path was then compared with every partial product, sum and carry of the RTL (Section 2.1). The result is the "Signal" column of Table 8.

#	Cell	Signal	Part of	Incr (ns)	Arr. (ns)
1	`dfrtp_1`	`a_r[1]`	launch FF	0.398	0.398
2	`clkinv_1`	$\overline{a_r[1]}$	AND gates	0.074	0.472
3	`o311ai_0`	merged	ANDs + fa1_1	0.348	0.820
4	`a21oi_1`	$c_2[1]$	fa2_0 carry	0.172	0.992
5	`maj3_1`	$c_2[2]$	fa2_1 carry	0.381	1.373
6	`xnor2_1`	$\overline{s_2[2]}$	fa2_2 sum	0.160	1.534
7	`maj3_1`	$\overline{c_3[2]}$	fa3_1 carry	0.385	1.918
8	`maj3_1`	$c_3[3]$	fa3_2 carry	0.377	2.295
9	`xnor3_1`	$s_3[3]$	fa3_3 sum	0.175	2.471
10	`dfrtp_1`	`p[6]`	capture FF	0.000	2.471

Table 8: Critical path at $T = 2.60$ ns (`sta/mult_array_timing.rpt`). fa*r_c* is the full adder in row *r*, column *c*; an overline marks a signal ABC built inverted. The capture flip-flop needs 0.134 ns of setup, so the required time is $2.600 - 0.134 = 2.466$ ns and the slack is $2.466 - 2.471 = -0.004$ ns.

Reading the path. Bit $a[1]$ leaves its register and enters the partial products. The `o311ai` merges partial-product ANDs with full adder fa1_1. Its sum $s_1[1]$ is passed down as $r_1[0]$ into fa2_0. The signal then ripples along the row-2 carry chain (fa2_0 $\to$ fa2_1 $\to$ fa2_2). The sum $s_2[2]$ of fa2_2 is passed down as $r_2[1]$ into fa3_1. Finally, the signal ripples along the row-3 carry chain (fa3_1 $\to$ fa3_2 $\to$ fa3_3) to the top sum bit $s_3[3] = p[6]$. ABC built some carries and sums inverted, because inverting CMOS gates are smaller and faster; the next gate absorbs the inversion.

Comparison with $2N$. The path crosses **7 full-adder stages**: fa1_1, fa2_0, fa2_1, fa2_2, fa3_1, fa3_2 and fa3_3. Six of the eight gates compute a full-adder carry or sum directly, and the `o311ai` contains fa1_1. Seven stages agree well with the estimate of about $2N = 8$ full-adder delays, which is only an approximation of the longest chain. The 8 gates contribute 2.073 ns of logic delay ($2.471 - 0.398$), or about 0.30 ns per stage. Each stage costs a single gate because the path needs either the carry or the sum of each adder, never both: a carry is one `maj3` (or AOI) cell and a sum is one `xnor` cell.

Other paths. The input-to-register paths (1.0 ns input delay straight into a flip-flop) and register-to-output paths (CLK→Q plus 1.0 ns output delay) are short. The asynchronous-reset *recovery* check, which requires the reset to be released a minimum time before the clock edge, has +9.228 ns of slack at 10 ns and +1.828 ns at 2.60 ns. Neither limits the clock.

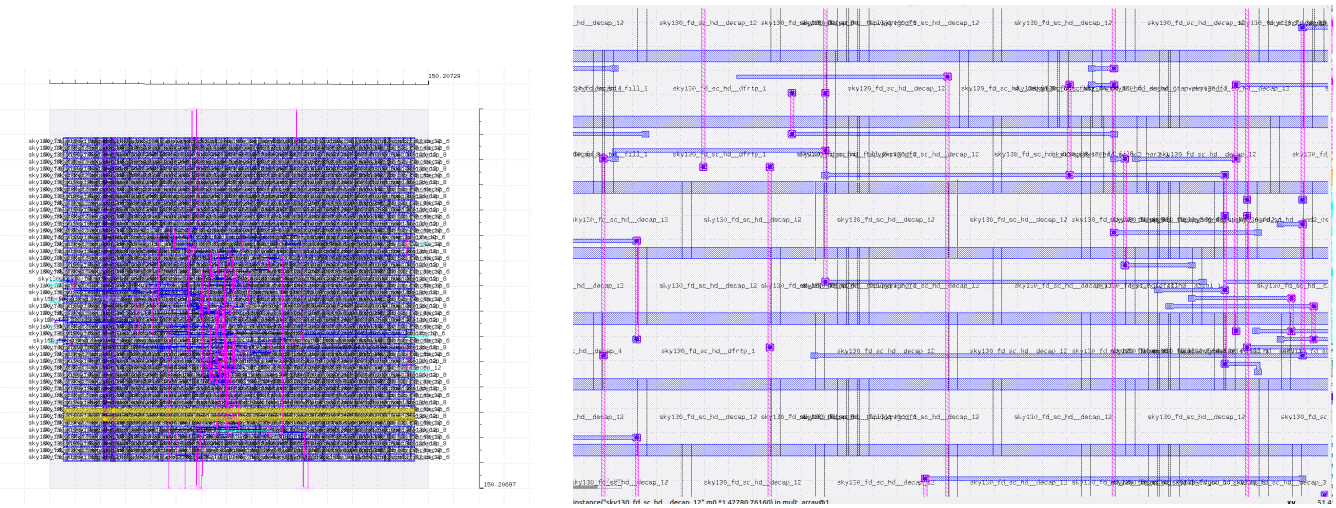
5 Physical Design (Part D)

5.1 The flow

LibreLane turns the RTL into a manufacturable layout in about 80 automatic steps. It was run with `--pdk sky130A --scl sky130_fd_sc_hd`, because the container's default PDK is a different one. Table 9 explains the main stages.

`pnr/config.json` contains exactly the required keys:

- `DESIGN_NAME = "mult_array"`
- the two RTL files
- `CLOCK_PORT = "clk"`



(a) whole die (b) zoomed view

Figure 3: Final layout in KLayout. (a) The 150 μm × 150 μm die: 47 standard-cell rows filled with tap, decap and filler cells, with the routed logic in the centre and wires running to the I/O pins on the edges. (b) Zoomed view: the VPWR/VGND rails bound each row, which holds standard cells (e.g. `dfrtp_1`, `decap_12`). Metal wires and vias connect the cell pins.

Stage	What it does
Synthesis	RTL to standard cells, as in Part B
Floorplan	Fixes the die (whole chip) at 150 μm × 150 μm and the core (where cells go) at 17 759.5 μm^2; creates 47 placement rows
Power grid	Lays out the VPWR (1.8 V) and VGND (0 V) supply rails
Tap/decap	Inserts well-tap and decoupling cells
Placement	Puts every cell at a legal site in the rows
CTS	Clock-tree synthesis: buffers the clock so it reaches all 16 flip-flops at nearly the same time; the difference in arrival is the *skew*
Timing repair	Resizes gates and inserts buffers to fix setup, slew and load violations
Routing	Draws the metal wires and vias between pins
Extraction	Computes each wire's resistance and capacitance (parasitics)
Signoff	STA with real wires at 9 corners; DRC (Magic, KLayout); LVS (Netgen); antenna and IR-drop checks
GDSII	Writes the final layout file sent to the foundry

Table 9: Main stages of the LibreLane Classic flow.

- `CLOCK_PERIOD`
- `FP_SIZING = "absolute"`
- `DIE_AREA = [0, 0, 150, 150]`

Corners. Manufacturing and operating conditions vary, so signoff STA is run at 9 *corners*:

- three process/voltage/temperature (PVT) conditions: typical (tt, 1.80 V, 25 °C), slow (ss, 1.60 V, 100 °C) and fast (ff, 1.95 V, −40 °C);
- each combined with minimum, nominal and maximum wire RC.

The chip must meet timing at all of them. Setup is hardest at the slow corner, and hold at the fast corner.

5.2 Timing closure

The first period is $1.2 \times T_{\min} = 1.2 \times 2.604$ ns $= 3.125$ ns, rounded up to the next 0.5 ns, which is 3.5 ns. The 20 % margin over $T_{\min}$ allows for wires and the clock tree. After each run that failed setup, the period was increased by 0.5 ns. Table 10 lists all 10 runs; 8.0 ns is the first period that passes.

Run	Period (ns)	Setup slack (ns)	Hold slack (ns)	Wire-length (μm)	Result
1	3.5	−2.184	+0.114	2674	FAIL
2	4.0	−1.684	+0.133	2674	FAIL
3	4.5	−1.184	+0.135	2675	FAIL
4	5.0	−0.682	+0.135	2675	FAIL
5	5.5	−0.208	+0.135	2688	FAIL
6	6.0	−0.154	+0.135	2612	FAIL
7	6.5	−0.224	+0.136	2568	FAIL
8	7.0	−0.137	+0.136	2560	FAIL
9	7.5	−0.100	+0.136	2499	FAIL
10	**8.0**	**+0.400**	+0.136	2499	**PASS**

Table 10: All LibreLane runs. Slack is the worst over the 9 signoff corners. Every run has 0 DRC errors and 0 LVS errors; FAIL means setup slack < 0 ns.

Why the slack changes unevenly. The timing-repair step runs after clock-tree synthesis but *before* routing, and only aims for a 0.05 ns setup margin. Routing then adds real wire delay, which takes some slack back at the slow corner. This explains three regimes in Table 10:

- **3.5 to 5.0 ns:** repair cannot close timing (at 3.5 ns it reaches only −2.02 ns before routing). Nothing else changes between runs, so each 0.5 ns increase in the period improves the final slack by exactly 0.5 ns.
- **5.5 to 7.5 ns:** repair can close timing, but only just. At 6.0 ns it reaches +0.068 ns before routing; routing then costs 0.22 ns, leaving −0.154 ns (Listing 3). All runs in this range end in a narrow band between −0.10 ns and −0.22 ns.
- **8.0 ns:** the design meets timing before routing without any repair, so real margin is left (+0.400 ns).

The synthesised logic is identical in every run (16 flip-flops, 42 multi-input cells, 4 inverters). Only the number of buffers changes, from 33 at 3.5 ns to 21 at 8.0 ns (including the 17 I/O port buffers, which are present in every run). Because the failing runs are so close to 0 ns, small differences in tool

version, netlist or placement can move the passing period by 0.5 to 1.5 ns.

Metric	Value
CLOCK_PERIOD used	8.0 ns
Number of runs needed	10 runs
Core area	17 759.5 μm^2 (die 22 500 μm^2)
Cells after synthesis (Part B)	63 cells
Cells after place and route	1883 instances: 86 logic, flip-flop and buffer cells, 245 tap cells, 1552 decap and filler cells
Kinds of cells added	`buf_2` (17 I/O port buffers), `clkdlybuf4s*` (4 repair buffers), `clkbuf_16` (3 clock buffers), `tapvpwrvgnd_1` (245), `decap_*` (1298), `fill_*` (254)
Wirelength	2499 μm (561 vias)
Setup slack	+0.400 ns (corner `max_ss_100C_1v60`)
Post-route $F_{\max}$	$1000/(8.0-0.400) = 131.58$ MHz
$F_{\max}$ from Part C	384.02 MHz
DRC errors	0 errors (Magic and KLayout)
LVS errors	0 errors (Netgen)

Table 11: Part D results for the final run (from `pnr/metrics.json`).

5.3 Cells added by place and route

Synthesis produced 63 cells, but the final layout has 1883 instances. LibreLane re-synthesises the RTL with its own script, which gives 62 cells. It then adds 17 I/O port buffers, 4 repair buffers and 3 clock buffers, so the layout has $62+17+4+3 = 86$ functional cells. Everything else is support cells, as Table 12 shows. The cells occupy only 1264.96 μm^2 of the 17 759.5 μm^2 core (7.1 % utilisation), so most of the core is filled with filler and decap cells.

Class	Count	Area (μm^2)	Why it is needed
I/O port buffers	17	85.1	One `buf_2` on each of the 9 inputs (`rst_n`, `a`, `b`) and 8 outputs (`p`); isolates the internal logic from unknown external loads and drivers
Repair buffers	4	40.0	`clkdlybuf4s*`; split heavily loaded nets (most likely `rst_n`, which drives all 16 flip-flops) to fix slew and load limits
Clock buffers	3	75.1	Drive 16 clock pins with sharp edges and low skew
Tap cells	245	306.5	Tie the n-well to VPWR and the substrate to VGND, preventing latch-up
Decap cells	1298	16 494.6	Charge reservoirs between VPWR and VGND that reduce supply noise
Filler cells	254		Keep wells, implants and power rails continuous across gaps (needed for DRC)

Table 12: Cells added by the flow, with their total area and purpose.

5.4 Signoff

Table 11 gives the required results, and Table 13 shows the slack at each corner.

- Setup is limited by the slow corner (+0.400 ns), and hold by the fast corner (+0.136 ns).
- The typical corner has 4.2 ns of setup slack to spare.
- Clock skew is about 0.26 ns.
- There are 0 antenna violations and 0 power-grid violations.
- The worst IR drop (voltage lost along the power rails) is 0.31 mV.
- Total power at the typical corner is 0.296 mW.
- The flow reports 11 max-slew warnings at the slow corners. These are not setup or hold violations.

Fig. 3 shows the final layout.

Corner	RC	Setup (ns)	Hold (ns)	$F_{\max}$ (MHz)
tt, 1.80 V, 25 °C	min	4.267	0.360	267.9
	nom	4.240	0.370	265.9
	max	4.209	0.377	263.8
ss, 1.60 V, 100 °C	min	0.500	0.945	133.3
	nom	0.449	0.963	132.4
	max	**0.400**	0.974	**131.6**
ff, 1.95 V, −40 °C	min	5.535	**0.136**	405.8
	nom	5.526	0.143	404.2
	max	5.519	0.147	403.1

Table 13: Setup and hold slack at the 9 signoff corners of the final run (period 8.0 ns). $F_{\max} = 1000/(8.0 - \text{setup slack})$.

6 Answers to Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic, and why is it not 16 + 12?

Synthesis produced $63-16 = 47$ logic cells (340.3 μm^2), not 28. "AND gate" and "full adder" are RTL building blocks, not single library cells. With `-flatten`, the module boundaries disappear and ABC re-implements the whole Boolean function with whatever cells give a good area and delay. Four effects explain the count:

- **A full adder needs at least two cells:** a sum cell (`xor2`, `xnor2`, `xor3` or `xnor3`; 10 in total) and a carry cell (`maj3`) or an equivalent NAND/AOI network. That alone is about 24 cells for 12 full adders.
- **AND gates are absorbed** into compound gates such as `a21oi`, `a22oi`, `o311ai`, `nand3` and `nand4`. Only 6 plain AND cells remain.
- **Constant inputs simplify some adders.** The carry into every row ($c_i[0] = 0$) and the top bit of r_0 are 0, so those full adders become half adders (an `xor2` plus a `nand2` or `and2`).
- **Inverting cells need a few inverters** (`clkinv`, `nand2b`, `nor2b`).

Q2. What is the critical path, how many cells are on it, and how does it compare with about 2N full-adder delays?

It runs from input register `a_r[1]` (`_095_`) to output register `p[6]` (`_092_`). It has 10 cells: the launching flip-flop, 8 gates (`clkinv`, `o311ai`, `a21oi`, `maj3`, `xnor2`, `maj3`, `maj3`, `xnor3`) and the capturing flip-flop. The data arrives after 2.471 ns, giving $T_{\min} = 2.604$ ns. Matching every net to the RTL by truth table (Table 8) shows that the path crosses **7 full-adder stages**: along the row-2 carry chain, down through $s_2[2]$, and along the row-3 carry chain to $s_3[3] = p[6]$. Six of the 8 gates

compute a full-adder carry or sum directly. Seven stages are close to the estimate of $2N = 8$ full-adder delays. Each stage costs only one gate (about 0.30 ns), because the path uses either the carry or the sum of each adder, not both.

Q3. What percentage of the cell area is flip-flops, and why is it so large?

Flip-flops take 400.38 μm^2 of 740.71 μm^2, which is 54.05 %, although they are only 16 of the 63 cells (25 %). There are two reasons:

- **A flip-flop is a large cell.** One `dfrtp_1` is 25.02 μm^2: 6.7 times a `nand2_1` (3.75 μm^2) and 3.5 times the average logic cell (7.24 μm^2). An edge-triggered flip-flop with asynchronous reset is two latches plus clock buffering and reset logic, a few dozen transistors against 4 for a NAND2.
- **Scaling.** The array logic grows as N^2 (16 ANDs and 12 adders), while the registers grow only as $4N$ (16 flip-flops). At $N = 4$ the logic is still small, so the registers dominate. At 8 bits the share falls to about 35 % (Q6).

Q4. Why is the post-route $F_{\max}$ lower than in Part C? Which cells did the flow add, and why?

The post-route $F_{\max}$ is 131.58 MHz, against 384.02 MHz from Part C, about 2.9 times lower. The two numbers are measured under very different conditions:

1. **Corner.** Part C used only the typical library. Signoff reports the worst of 9 corners, which is slow-slow at 1.60 V and 100 °C, where transistors are about half as fast. In the same run, the typical corner alone allows $1000/(8.0 - 4.240) \approx 266$ MHz.
2. **Wires.** Part C had no parasitics. After routing, 2499 μm of wire and 561 vias add resistance and capacitance to every net, and the pins sit on the edge of a 150 μm die.
3. **Clock tree.** Part C assumed an ideal clock. The real tree has insertion delay and about 0.26 ns of skew.
4. **Constraints.** LibreLane's constraints are stricter: input and output delays are 20 % of the period (1.6 ns at 8.0 ns, against 1.0 ns in Part C), plus 0.25 ns of clock uncertainty and a 5 % timing derate.
5. **Netlist.** LibreLane re-synthesises the design (62 cells instead of 63) and inserts buffers, so the post-route critical path is not the same as in Part C.

The added cells and their purposes are listed in Table 12: I/O port buffers, repair buffers, clock buffers, tap cells, decap cells and filler cells.

Q5. What do DRC and LVS verify, and why must both pass?

DRC (design rule check) verifies the *geometry* of the layout against the foundry's rules for every layer: minimum widths, spacings, enclosures, overlaps, density and antenna ratios. These rules describe what the factory can print reliably, and a violation can cause shorts, opens or very low yield.

LVS (layout versus schematic) extracts the transistors and their connections from the layout and checks that they form exactly the circuit in the netlist: same devices, same nets, same pins.

The two checks cover different risks. A layout can be manufacturable but the wrong circuit, or the right circuit but impossible to manufacture. A mask set is expensive and cannot be fixed after tape-out, so both must pass first. Our layout has 0 DRC errors in Magic and KLayout (the router's 6 first-pass violations were fixed in its second pass), 0 LVS errors in Netgen, and 0 antenna violations.

Q6. Estimate how the full adders and critical path change at 8 bits.

Table 14 applies the formulas from the assignment to $N = 8$.

Quantity	Formula	$N = 4$	$N = 8$	Change
AND gates	N^2	16	64	$\times 4$
Full adders	$N(N-1)$	12	56	$\times 4.67$
Flip-flops	$4N$	16	32	$\times 2$
Critical path	$2N$ FA delays	8	16	$\times 2$

Table 14: Scaling from 4 to 8 bits (counts in cells; path length in full-adder delays).

From Part C, the critical path crosses 7 full-adder stages in 2.073 ns of logic, so one stage costs $2.073\text{ ns}/7 = 0.296$ ns. The flip-flop overhead is $0.398\text{ ns} + 0.134\text{ ns} = 0.532$ ns (CLK→Q plus setup). Using the $2N = 16$ estimate for $N = 8$:

$$T_{\min} \approx 0.532 + 16 \times 0.296 = 5.27 \text{ ns},$$
$$F_{\max} \approx 1000/5.27 = 190 \text{ MHz}.$$

The number of full adders grows quadratically, by 4.7 times, but the critical path grows only linearly, by about 2 times, so $T_{\min}$ roughly doubles (2.604 ns → 5.27 ns). If the longest chain keeps the measured $2N - 1$ form (7 stages for $N = 4$, 15 for $N = 8$), the estimate is $0.532 + 15 \times 0.296 = 4.97$ ns (201 MHz).

Scaling the logic area by the gate count, $(56 + 64)/(12 + 16) \approx 4.3$, gives about 1460 μm^2 of logic. With 32 flip-flops (about 800 μm^2), the total is about 2260 μm^2, and the flip-flop share falls to about 35 %.

7 Conclusion

The 4-bit array multiplier was taken from RTL to a DRC- and LVS-clean GDSII layout:

- **Part A:** the exhaustive testbench passes all 256 vectors and catches an injected adder bug (95 errors). The hand-worked example $9 \times 7 = 63$ appears in the waveform with a 2-cycle latency.
- **Part B:** synthesis gives 63 cells and 740.71 μm^2, with flip-flops taking 54.05 % of the area. The gate-level netlist passes the same testbench.
- **Part C:** STA gives $T_{\min}$ = 2.604 ns (384.02 MHz) at the typical corner. The critical path from `a_r[1]` to `p[6]` matches the $2N$ full-adder estimate.
- **Part D:** place and route closes timing at 8.0 ns after 10 runs (+0.400 ns slack, 131.58 MHz), with 0 DRC and 0 LVS errors.

The main lesson is how much the operating conditions matter. The same logic runs at 384 MHz in an idealised typical-corner analysis, at about 266 MHz at the typical corner after layout, and at 132 MHz at the slow corner the chip must also survive.

Use of AI tools. An AI assistant was used to check the deliverables against the assignment, analyse the tool logs and metrics, and draft and typeset this report from our results. All

numbers come from our own tool runs, and we have checked every part of the report.

Individual Contributions

The work was divided equally between the two team members. Following the assignment's marking scheme, each member was responsible for 50 of the 100 marks: Parts A and C (30 + 20 marks) and Parts B and D (20 + 30 marks).

Bhargav (2023112014) designed and verified the RTL in Part A. This covers the full adder, the array multiplier, the exhaustive self-checking testbench, the error-injection test, the hand-worked example and the waveform. He carried out the static timing analysis in Part C: he wrote `sta.tcl`, found $T_{\min}$ and $F_{\max}$, and traced the critical path through the full-adder stages. He answered Q2 and Q6, maintained the README command log, and wrote and typeset this report.

Nitin (2023112026) carried out the logic synthesis in Part B: he wrote `synth.tcl`, ran the gate-level simulation and analysed the cell usage and area. He also completed the physical design in Part D. This covers `config.json`, the 10 LibreLane runs and their timing-closure analysis, the analysis of the cells added by the flow, DRC/LVS signoff and the layout screenshots. He drew the block diagram, answered Q1, Q3, Q4 and Q5, and reviewed the report and cross-checked its results.

A Logs and Detailed Tables

```
MISMATCH: a=3 b=3 expected=9 got=5
MISMATCH: a=3 b=6 expected=18 got=10
MISMATCH: a=3 b=7 expected=21 got=9
MISMATCH: a=3 b=11 expected=33 got=29
MISMATCH: a=3 b=12 expected=36 got=20
... (90 more MISMATCH lines)
FAIL: 256 vectors checked, 95 errors
```

Listing 1: Error test (`sum = a ^ b`): first mismatches and summary.

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:49: $finish called at 7700000 (1ps)
```

Listing 2: Gate-level simulation output (`report/partB/gls.log`).

```
[INFO] Setting output delay to: 1.2
[INFO] Setting input delay to: 1.2
[INFO] Setting clock uncertainty to: 0.25
[INFO] Setting timing derate to: 5.0%
+ repair_timing -verbose -setup -setup_margin 0.05 ...
[INFO RSZ-0094] Found 3 endpoints with setup violations.
 Iter | ... |   WNS  | TNS  | Viol | Worst
    0 | ... | -1.335 | -3.2 |    3 | _90_/D
final | ... |  0.068 |  0.0 |    0 | _90_/D
```

Listing 3: LibreLane timing repair before routing, 6.0 ns run: timing is closed to +0.068 ns; the final slack after routing is −0.154 ns.

Group	Cell	Count	Area (μm^2)
Flip-flops	`dfrtp_1`	16	400.384
Sum	`xnor2_1`	4	35.034
	`xor2_1`	3	26.275
	`xnor3_1`	2	45.043
	`xor3_1`	1	23.773
Carry	`maj3_1`	3	30.029
AND	`and2_0`	2	12.512
	`and3_1`	1	6.256
	`and4_1`	1	8.758
	`lpflow_isobufsrc_1`	2	12.512
NAND/NOR	`nand2_1`	8	30.029
	`nand2b_1`	1	6.256
	`nand3_1`	3	15.014
	`nand4_1`	1	6.256
	`nor2_1`	1	3.754
	`nor2b_1`	1	6.256
	`nor3_1`	1	5.005
AOI/OAI	`a21oi_1`	6	30.029
	`a22oi_1`	1	7.507
	`o21ai_0`	1	5.005
	`o311ai_0`	1	8.758
	`o2111ai_1`	1	8.758
Inverters	`clkinv_1`	2	7.507
Total	23 cell types	**63**	**740.710**

Table 15: All cells in the synthesised netlist (`synth/mult_array_stat.rpt`).

```
Startpoint: _095_ (rising edge-triggered flip-flop clocked by
      clk)
Endpoint: _092_ (rising edge-triggered flip-flop clocked by clk
      )
Path Group: clk
Path Type: max

   Delay     Time   Description
---------------------------------------------------------
   0.000    0.000   clock clk (rise edge)
   0.000    0.000   clock network delay (ideal)
   0.000    0.000 ^ _095_/CLK (sky130_fd_sc_hd__dfrtp_1)
   0.398    0.398 ^ _095_/Q (sky130_fd_sc_hd__dfrtp_1)
   0.074    0.472 v _039_/Y (sky130_fd_sc_hd__clkinv_1)
   0.348    0.820 ^ _047_/Y (sky130_fd_sc_hd__o311ai_0)
   0.172    0.992 v _050_/Y (sky130_fd_sc_hd__a21oi_1)
   0.381    1.373 v _063_/X (sky130_fd_sc_hd__maj3_1)
   0.160    1.534 v _064_/Y (sky130_fd_sc_hd__xnor2_1)
   0.385    1.918 v _066_/X (sky130_fd_sc_hd__maj3_1)
   0.377    2.295 v _075_/X (sky130_fd_sc_hd__maj3_1)
   0.175    2.471 v _079_/X (sky130_fd_sc_hd__xnor3_1)
   0.000    2.471 v _092_/D (sky130_fd_sc_hd__dfrtp_1)
            2.471   data arrival time

   2.600    2.600   clock clk (rise edge)
   0.000    2.600   clock network delay (ideal)
   0.000    2.600   clock reconvergence pessimism
            2.600 ^ _092_/CLK (sky130_fd_sc_hd__dfrtp_1)
  -0.134    2.466   library setup time
            2.466   data required time
---------------------------------------------------------
            2.466   data required time
           -2.471   data arrival time
---------------------------------------------------------
           -0.004   slack (VIOLATED)
```

Listing 4: OpenSTA critical-path report at $T = 2.60$ ns (`sta/mult_array_timing.rpt`; the reset recovery check, slack +1.828 ns, is omitted).